

NCI TRADING BRAIN

Complete Build Specification & Code

For Claude Code Development Team

Document Version:	1.0
Build Date:	June 03, 2026
System Name:	NERDCOMMAND Command Intelligence
Organization:	GangsterNerds LLC

TABLE OF CONTENTS

- 1. Executive Summary
- 2. Current Build Status
- 3. System Architecture
- 4. Four-Layer Design
- 5. Five-Stage Data Pipeline
- 6. Trading Strategies
- 7. Python Core Modules
- 8. API Endpoints
- 9. Configuration Guide
- 10. Deployment Roadmap
- 11. Technical Stack

1. EXECUTIVE SUMMARY

Mission

NCI (NERDCOMMAND Command Intelligence) is a modular trading system designed to generate consistent income (\$300+/month target) through algorithmic trading, options strategies, and dividend income optimization. Current phase: Foundation Layer (Phase 1).

Key Components

- Tradier API integration for real-time market data and order execution
- Signal generation engine (impulse system: EMA + MACD)
- Risk management rules and position sizing
- Four-layer modular architecture (Data / Analysis / Execution / Monitoring)
- Real-time trading dashboard with manual override capability
- Paper trading environment for strategy validation

2. CURRENT BUILD STATUS

Component	Status	Progress
Tradier API Integration	In Progress	60%
Market Data Pipeline	In Progress	40%
Signal Generation	Planned	0%
Risk Management	Planned	0%
Trading Dashboard	Planned	0%
Paper Trading	Planned	0%

3. SYSTEM ARCHITECTURE

Four-Layer Architecture Design

NCI uses a layered architecture that enables independent optimization and clear separation of concerns:

Layer	Purpose	Components
Data Layer	Ingest & normalize market data	Tradier API, TradingView, feeds
Analysis Layer	Generate signals	Indicators, ML models, filters
Execution Layer	Place & manage orders	Order router, position tracker
Monitoring Layer	Track & alert	Dashboard, P&L tracker, logs

Five-Stage Data Pipeline

All market data flows through 5 stages before execution: (1) **INGEST** - Real-time feeds from Tradier/TV, (2) **NORMALIZE** - Standardize formats, (3) **ANALYZE** - Apply indicators/ML, (4) **SIGNAL** - Generate scores, (5) **EXECUTE** - Risk-checked execution.

4. TRADING STRATEGIES

Strategy 1: The Wheel (Income Primary)

Sell CSP at 0.25 delta (30 DTE). If assigned, sell CC at 0.30 delta. Roll before expiration. Target: \$200-400/month per \$10k. Best for: SOFI, INTC, SLV, AAPL, AMZN (positive FCF).

Strategy 2: High-Yield Income ETFs

Allocation A: YMAX - weekly distributions, enable DRIP. **Allocation B:** DGRO - lower yield (~2%) + NAV appreciation. Recommended for accumulation phase.

Strategy 3: Swing Trading

Vehicle: SOFI. Entry: \$17.80-18.50 (pullback). Target: \$21-22. Stop: \$16.50. Hold: 3-5 days. CRITICAL: Exit before earnings (April 29 hard stop).

Strategy 4: Carry Trade Monitoring

Watch JPY/USD for unwinding signals (rising yen = broad market selloff risk). Use as macro overlay for SPY options.

5. PYTHON CORE MODULES - CODE EXAMPLES

Module 1: Tradier Client

```
class TradierClient:
    def __init__(self, api_token, sandbox=True):
        self.api_token = api_token
        self.base_url = "https://sandbox.tradier.com/v1" if sandbox else "https://api.tradier.com/v1"

    def get_quotes(self, symbols):
        url = f"{self.base_url}/markets/quotes"
        params = {"symbols": ",".join(symbols)}
        return requests.get(url, headers=self.headers, params=params).json()

    def get_positions(self):
        url = f"{self.base_url}/accounts/self/positions"
        return requests.get(url, headers=self.headers).json()

    def place_order(self, symbol, qty, order_type, price=None):
        payload = {"symbol": symbol, "quantity": qty, "type": order_type}
        if price:
            payload["price"] = price
        url = f"{self.base_url}/accounts/self/orders"
        return requests.post(url, headers=self.headers, json=payload).json()
```

Module 2: Signal Engine

```
class SignalEngine:
    def calculate_emas(self, prices):
        ema13 = pd.Series(prices).ewm(span=13).mean()
        ema26 = pd.Series(prices).ewm(span=26).mean()
        return ema13.values, ema26.values

    def calculate_macd(self, prices):
        ema12 = pd.Series(prices).ewm(span=12).mean()
        ema26 = pd.Series(prices).ewm(span=26).mean()
        macd = ema12 - ema26
        signal = macd.ewm(span=9).mean()
        histogram = macd - signal
        return macd.values, signal.values, histogram.values

    def generate_signal(self, prices):
        ema13, ema26 = self.calculate_emas(prices)
        macd, signal, hist = self.calculate_macd(prices)
        ema_rising = ema13[-1] > ema26[-1]
        macd_positive = hist[-1] > 0

        if ema_rising and macd_positive:
            return SignalType.BUY
        elif not ema_rising and not macd_positive:
            return SignalType.SELL
        else:
            return SignalType.HOLD
```


6. API ENDPOINTS & CONNECTIONS

GET /api/quotes

Real-time quotes for symbols

GET /api/positions

Current positions & unrealized P&L;

GET /api/signals

Latest trading signals

POST /api/orders/place

Execute new trade

GET /api/dashboard

Full dashboard state

GET /api/pnl/today

Today's P&L; summary

7. CONFIGURATION & SETUP

Environment Variables (.env)

```
TRADIER_SANDBOX_TOKEN=your_sandbox_token
TRADIER_PRODUCTION_TOKEN=your_prod_token
FLASK_ENV=development
FLASK_PORT=5000
DATABASE_URL=sqlite:///nci_trading.db
PAPER_TRADING_MODE=True
LOG_LEVEL=INFO
```

Installation Steps

- Clone repository to local machine
- Create Python 3.9+ virtual environment
- Install dependencies: `pip install -r requirements.txt`
- Create .env file with tokens
- Initialize database: `python manage.py db upgrade`
- Launch: `python app.py`
- Access dashboard: `http://localhost:5000`

8. DEPLOYMENT ROADMAP

Phase	Timeline	Deliverables
Phase 1: Foundation	Weeks 1-4	Tradier API, data pipeline, signals, paper trading
Phase 2: Intelligence	Weeks 5-8	ML models, advanced filters, risk rules, dashboard
Phase 3: Trading	Weeks 9-12	Live execution, position tracking, alerts
Phase 4: Optimization	Weeks 13+	Performance tuning, new strategies, rebalancing

9. TECHNICAL STACK

Backend

- Python 3.9+
- Flask (REST API)
- SQLAlchemy (ORM)
- Pandas + NumPy
- scikit-learn (ML)

Frontend

- HTML5 + CSS3
- JavaScript vanilla
- Chart.js or D3.js
- Dark theme

Integrations

- Tradier API
- TradingView
- SQLite/PostgreSQL

Key Dependencies (requirements.txt)

```
Flask==2.3.0
SQLAlchemy==3.0.0
Requests==2.31.0
pandas==2.0.0
numpy==1.24.0
scikit-learn==1.2.0
python-dotenv==1.0.0
```

10. CRITICAL SUCCESS FACTORS

- ✓ Always test in Tradier Sandbox first. Never skip paper trading.
- ✓ Risk management gates are non-negotiable. Every trade must pass position/stop/heat checks.
- ✓ Modular design over monolith. Build in isolation, test, integrate incrementally.
- ✓ Log everything. Capture signals, execution, errors, and trade rationale.
- ✓ Iterate rapidly. Each trade is a learning opportunity. Update strategies based on results.

Reference Materials

See project files for: Trading Brain AI Agent Foundational Knowledge Base (options pricing, neural networks, forex), Trading Brain Architectural Framework (design decisions, validation checkpoints).